Week 3 Practice Problems

The Master Theorem (as stated in lecture): for a recurrence of the form

$$T(n) = aT(n/b) + \Theta(n^d) \text{ if } n \geq n_0, \qquad T(n) = \Theta(1) \text{ if } n < n_0,$$

where a, b, d, n_0 are constants with $a > 0$, $b > 1$, $d \geq 0$, $n_0 > 0$:

$$T(n) = \begin{cases} \Theta(n^d \log n) & \text{if } d = \log_b a \\ \Theta(n^{\log_b a}) & \text{if } d < \log_b a \\ \Theta(n^d) & \text{if } d > \log_b a \end{cases}$$

Problem 1 : Practice with the Master Theorem

For each recurrence below, identify the constants a, b, and d. Then compute $\log_b a$, compare it with d, determine which case of the Master Theorem applies, and state the corresponding asymptotic bound.

(a) $T(n) = 4T(n/2) + \Theta(n^2)$

(b) $T(n) = 3T(n/3) + \Theta(1)$

(c) $T(n) = 7T(n/2) + \Theta(n^2)$

(d) $T(n) = 2T(n/4) + \Theta(n)$

(e) $T(n) = 2T(n/2) + \Theta(n \log n)$

Solution. (a) $a = 4$, $b = 2$, $d = 2$.

- $\log_b a = \log_2 4 = 2$.
- Since $d = 2 = \log_2 4$, this is the case where $d = \log_b a$.
- $T(n) = \Theta(n^d \log n) = \Theta(n^2 \log n)$.

(b) $a = 3$, $b = 3$, $d = 0$ (since $\Theta(1) = \Theta(n^0)$).

- $\log_b a = \log_3 3 = 1$.
- Since $d = 0 < 1$, this is the case where $d < \log_b a$.
- $T(n) = \Theta(n^{\log_b a}) = \Theta(n^1) = \Theta(n)$.

(c) $a = 7$, $b = 2$, $d = 2$.

- $\log_b a = \log_2 7 \approx 2.807$.

- Since $d = 2 < \log_2 7 \approx 2.807$, this is the case where $d < \log_b a$.
- $T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$.

(d) $a = 2$, $b = 4$, $d = 1$.

- $\log_b a = \log_4 2 = 0.5$.
- Since $d = 1 > 0.5$, this is the case where $d > \log_b a$.
- $T(n) = \Theta(n^d) = \Theta(n)$.

(e) It's a trick! The Master Theorem does not apply here :) Here $a = 2$ and $b = 2$. However, the non-recursive cost is $\Theta(n \log n)$, which cannot be expressed as $\Theta(n^d)$ for any constant d.

Problem 2 : Algorithm Analysis using Master Theorem

Consider the following pseudocode for Karatsuba's fast integer multiplication algorithm, which multiplies two n-digit integers x and y:

```
def karatsuba(x, y, n):
    if n == 1:
        return x * y  # O(1) single digit multiplication

    m = n // 2
    # Split x and y into high and low arrays:
    x_high, x_low = split_digits(x, m)  # O(n) work
    y_high, y_low = split_digits(y, m)  # O(n) work

    p1 = karatsuba(x_high, y_high, m)
    p2 = karatsuba(x_low, y_low, m)
    p3 = karatsuba(x_high + x_low, y_high + y_low, m)

    result = p1 * (10**(2*m)) + (p3 - p1 - p2) * (10**m) + p2
    return result
```

(a) Formulate a recurrence relation $T(n)$ for the running time of `karatsuba` on n-digit numbers. State the number of recursive calls a, the subproblem size reduction factor b, and the non-recursive work exponent d.

(b) Identify a, b, and d, and apply the Master Theorem to find the tight asymptotic bound $T(n) = \Theta(\cdot)$ for Karatsuba's algorithm.

Solution. (a) The algorithm makes 3 recursive calls (to compute p_1, p_2, p_3), each on inputs of size approximately $n/2$. Splitting digits, performing single additions, and bit-shifting/padding zeroes take linear time $\Theta(n)$. Thus, the recurrence is:

$$T(n) = 3T(n/2) + \Theta(n)$$

(b) From the recurrence, we have $a = 3$, $b = 2$, and $d = 1$. Computing $\log_b a = \log_2 3 \approx$

1.585. Since $d = 1 < \log_2 3 \approx 1.585$, we have from the Master Theorem that $d < \log_b a$. Therefore, $T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$.

Problem 3 : Debugging divide and conquer

Recall in lecture we discussed an initial approach to the Maximum Sum Subarray problem where the divide and conquer approach was flawed. Before the fix we had: recursively find the max subarray sum of the left half, recursively find the max subarray sum of the right half, and return the larger of the two (never considering subarrays that span the middle).

Let $A = [-2, 1, -3, 4, -1, 2, 1, -5]$, split into left half $[-2, 1, -3, 4]$ and right half $[-1, 2, 1, -5]$.

(a) Find the maximum subarray sum within the left half alone, and within the right half alone. What does this initial approach return for A?

(b) Using the corrected algorithm from lecture (which additionally computes maxPrefix and maxSuffix), state which suffix of the left half and what prefix of the right half combine to give the actual maximum subarray sum of A.

(c) Now identify the actual maximum subarray sum of A, and state the subarray that gives this result.

Solution. **(a)** From all the subarrays, we see that the maximum sum (for the left side) is 4, which is given by [4]. For the right side, the maximum sum is 3, which is given by $[2, 1]$. The flawed approach returns $\max(4, 3) = 4$.

(b) The crossing subarray $[4, -1, 2, 1]$ splits exactly at the middle boundary into a suffix of the left half, [4] (sum 4), and a prefix of the right half, $[-1, 2, 1]$ (sum 2).

(c) The actual maximum subarray sum of A is 6, which is given by $[4, -1, 2, 1]$. This subarray starts at the last position of the left half and continues into the right half, so it belongs to neither of two exclusively.

Problem 4 : Tracing Karatsuba's algorithm

Use Karatsuba's algorithm to multiply the 4-bit numbers $x = 1101_2$ $(= 13)$ and $y = 0110_2$ $(= 6)$.

(a) Split x and y into their high and low halves: x_L, x_R, y_L, y_R (as bit-strings and as decimal values).

(b) Compute the following: $P_1 = x_L y_L$, $P_2 = x_R y_R$, and $P_3 = (x_L + x_R)(y_L + y_R)$.

(c) Compute $x_L y_R + x_R y_L$ using $P_3 - P_1 - P_2$, and check it against $x_L y_R + x_R y_L$.

(d) Combine everything with the equation $2^n P_1 + 2^{n/2}(P_3 - P_1 - P_2) + P_2$ (here $n = 4$) to get the final product, and verify it equals $x \cdot y$.

Solution. **(a)** $x = 1101_2$ splits into $x_L = 11_2 = 3$ and $x_R = 01_2 = 1$. $y = 0110_2$ splits into $y_L = 01_2 = 1$ and $y_R = 10_2 = 2$.

(b) $P_1 = x_L y_L = 3 \cdot 1 = 3$. $P_2 = x_R y_R = 1 \cdot 2 = 2$. $x_L + x_R = 4$, $y_L + y_R = 3$, so $P_3 = 4 \cdot 3 = 12$.

(c) $P_3 - P_1 - P_2 = 12 - 3 - 2 = 7$. $x_L y_R + x_R y_L = 3 \cdot 2 + 1 \cdot 1 = 6 + 1 = 7$. These match.
(d)

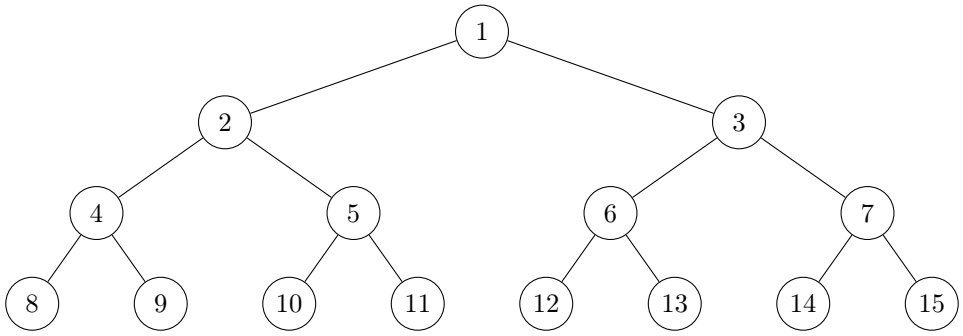
$$2^4 P_1 + 2^2(P_3 - P_1 - P_2) + P_2 = 16 \cdot 3 + 4 \cdot 7 + 2 = 48 + 28 + 2 = 78$$

$$x \cdot y = 13 \cdot 6 = 78$$

They match.

Problem 5 : Heap structure

Consider the array $A = [78, 61, 70, 57, 39, 32, 66, 26, 23, 29]$, viewed as a heap: index 1 is the root, and index i's left and right children are stored at indices $2i$ and $2i + 1$ (when those indices are ≤ 10). The diagram below is just to help visualize node indices (not values). You can use it for your reference.



(a) For each of nodes 1 through 5, state the value stored there, and the values stored at its left and right children (if they exist). Which nodes, among all 10, have no children at all?

(b) Double check that A satisfies the max-heap property (every node's value is $\geq$ the values of both its children) for every node.

(c) What is the height of this heap (the number of edges on the longest root to leaf path)?

Solution. (a) Node 1 (value 78): children at indices $2, 3$, values $61, 70$. Node 2 (value 61): children at indices $4, 5$, values $57, 39$. Node 3 (value 70): children at indices $6, 7$, values $32, 66$. Node 4 (value 57): children at indices $8, 9$, values $26, 23$. Node 5 (value 39): has only a left child, at index 10, value 29. Nodes $6, 7, 8, 9, 10$ have no children at all as these are the leaves.

(b) Node 1 (78) $\geq$ its children $(61, 70)$. Node 2 (61) $\geq$ its children $(57, 39)$. Node 3 (70) $\geq$ its children $(32, 66)$. Node 4 (57) $\geq$ its children $(26, 23)$. Node 5 (39) $\geq$ its one child (29). Every check holds, so A is a valid max heap.

(c) Node 1 is at depth 0, nodes $2, 3$ at depth 1, nodes $4, 5, 6, 7$ at depth 2, and nodes $8, 9, 10$ at depth 3. So the height is 3.